```
function depsi(param_file)


% depsi
%
% Usage: depsi('filename')
%
% This is the main program for a PS analysis based on a single
% master stack of interferograms. The ambiguity function,
% bootstrapping as well as integer least-squares can
% be used for the estimations.
%
% The program consists of a number of modules, which enables
% easy addition of steps and the formulation of alternatives.
%
% The modules are:
% - ps_selection:            reads the data in buffers and
%                            selects the psc, based on amplitude
%                            dispersion or high amplitude in a
%                            minimal number of scenes.
% - form_network:            forms a network between the psc
%                            by Delauney triangulation or a more
%                            redundant network.
% - psc_est:                 estimation of the ambiguities and
%                            parameters of psc by ambiguity
%                            function, bootstrapping or ils.
% - spatial_unwrapping:      spatial unwrapping of the network.
% - filtering:               estimation of atmospheric phase
%                            screen (APS) and non-linear defor-
%                            mation by filtering.
% - atmo_kriging:            atmosphere estimation by Kriging.
% - ps_est:                  estimation of the ambiguities and
%                            parameters of ps by ambiguity
%                            function, bootstrapping or ils.
%
% The program needs an input file (see manual
% for more information). Besides, you might need to change a few
% parameters at the beginning of this script.
%
% Using 'save [filename]' and 'load [filename]' you can easily save
% and load the workspace after a certain module. This prevents the
% need to re-run the whole program for a certain step.
%
% ------------------------------------------------------------------------
% File.............: depsi.m
% Version & Date..: 1.7.2.16, 12-DEC-2009
% Authors.........: Freek van Leijen
%                   Gini Ketelaar
%                   Petar Marinkovic
%                   Delft Institute of Earth Observation and Space Systems
%                   Delft University of Technology
% ------------------------------------------------------------------------
%
% This software is developed by Delft University of Technology and is
% intended for scientific use only. Applications for commercial use are
% prohibited.
%
% Copyright (c) 2004-2009 Delft University of Technology, The Netherlands
%
% Change log
% v1.7.2.8, Freek van Leijen
% - processing group 8: time series noise filter
% - calibration of full crop in one step
% -
```

```
% v1.7.2.12, Freek van Leijen
% - filenames in cells
% v1.7.2.17, Freek van Leijen
% - added global weighted_unwrap
% v1.7.4.0, Freek van Leijen
% - initiated ps_set_globals
% - added global run_mode
% v1.7.7.0, Freek van Leijen
% - added radarsat-2
% v1.7.7.4, Freek van Leijen
% - added inclusion of reference point in ps_selection.m
%



fprintf(1,'\n');
fprintf(1,'PS analysis has started....\n');

if exist('plots','dir')~=7
  mkdir('plots');
end
if exist('plots/atmospheric_filter','dir')~=7
  mkdir('plots/atmospheric_filter');
end
if exist('plots/noise_filter','dir')~=7
  mkdir('plots/noise_filter');
end

%set globals
ps_set_globals;

if exist(param_file)
  project_id = readinput('project_id',[],param_file);
else
  error('The parameter file you specified does not exist.');
end

processing_groups = readinput('processing_groups',[],param_file);
if isempty(processing_groups)
  processing_groups = 1:8;
end


if find(processing_groups == 1)

  ps_readinput_parameters; % script to read input parameters
  if isempty(processing_groups)
    processing_groups = 1:8;
  end

  % ------------------------------------------------------------------------
  % 1
  % Initialization
  % ------------------------------------------------------------------------


  % Counters
  % ------------------------------------------------------------------------

  fig = 0; %counter of figures
  detrend_flag = 'no';
  defo_model_flag = 'no';
  Nref = repmat(NaN,Npsc_selections,1);
```

```
  % General parameters
  % --------------------------------------------------------------------

  if strcmp(sensor,'ers')
    lambda = 0.0565646;    % wavelength [m]
    sat_vel = 7550;        % satellite velocity [m/s]
  elseif strcmp(sensor,'asar')
    lambda = 0.0562356;    % wavelength [m]
    sat_vel = 7550;        % satellite velocity [m/s]
  elseif strcmp(sensor,'rsat1')
    lambda = 0.0564105;    % wavelength [m] carrier_f=5300432000 Hz, c=2.99e8
    sat_vel = 7550;        % satellite velocity [m/s]
  elseif strcmp(sensor,'rsat2')
    lambda = 0.055465772433;    % wavelength [m]
    sat_vel = 7550;        % satellite velocity [m/s]
  elseif strcmp(sensor,'tsx')
    lambda = 0.031000;     % wavelength [m]
    sat_vel = 7600; %???   % satellite velocity [m/s]
  elseif strcmp(sensor,'Sentinel-1')
    lambda = 0.031000;     % wavelength [m]
    sat_vel = 7590; %???   % satellite velocity [m/s]
  else
    error('The sensor you specified is not supported yet');
  end

  m2ph = -4*pi/lambda; % meters to phase constant
  %toolbox_version = '1.7.2.16, 12-DEC-2009';
  %toolbox_version = depsiversion(); % dynamically get current version from SVN



  % Model parameters
  % --------------------------------------------------------------------

  Npar_max = ps_model_definitions('Npar_max');

  if length(final_model)>1
    warning('You specified multiple final models. Using the first.');
    final_model = final_model(1);
  end

  if isempty(std_param)
    std_param = [40 0.1 2 0.02 0.01 0.01 0.02];
    % topo, master_atmo, subpixel, linear, quadratic, cubic, periodic
  end


  % Read input
  % --------------------------------------------------------------------

%input_file='/home/wuwenhao/data_disk/Shanxi/data_processing/ps_toolbox/ps_toolb
ox/templates/input_file.txt'
  disp(input_file)
  [orbitnr,dates,filenames_slc,filenames_ifgs,filenames_h2ph, ...
   filenames_output,Bperp,Btemp,Bdop,l0,lN,p0,pN] = ...
    textread(input_file,'%s %s %s %s %s %s %f %f %f %f %f %f %f');

   disp(filenames_slc)
  crop_in = [l0 lN p0 pN];
  orbitnr = strvcat(orbitnr);
  dates = strvcat(dates);
```

```
  if strcmp(filenames_ifgs(end),'dummy')
    filenames_ifgs(end) = [];
    filenames_h2ph(end) = [];
    filenames_output(end) = [];
    Bperp(end) = [];
    Btemp(end) = [];
    Bdop(end) = [];
  end

  Nslc = size(filenames_slc,1);
  Nifgs = size(filenames_ifgs,1);

  [Btemp,Btemp_index] = sort(Btemp);
  orbitnr(1:Nifgs,:) = orbitnr(Btemp_index,:);
  dates(1:Nifgs,:) = dates(Btemp_index,:);
  filenames_slc(1:Nifgs) = filenames_slc(Btemp_index);
  filenames_ifgs = filenames_ifgs(Btemp_index);
  filenames_h2ph = filenames_h2ph(Btemp_index);
  filenames_output = filenames_output(Btemp_index);
  Bperp = Bperp(Btemp_index);
  Bdop = Bdop(Btemp_index);
  crop_in(1:Nifgs,:) = crop_in(Btemp_index,:);

  %find master index
  master_index = find(Btemp<0);
  master_index = master_index(end)+1;

  switch run_mode
    case 'validation'

      if exist('validation','dir')~=7
        mkdir('validation');
      end

      orbitnr_valid = [orbitnr(1:master_index-1,:);...
                       orbitnr(end,:);...
                       orbitnr(master_index:end-1,:)];
      Btemp_valid = round(365*[Btemp(1:master_index-1);...
                          0; ...
                          Btemp(master_index:end)]);

      ifgs_valid = [repmat(master_index,Nifgs,1) [1:master_index-1
master_index+1:Nslc]'];

      orbitnr_valid_fid = fopen(['validation/orbitnr_valid.txt'],'w');
      fprintf(orbitnr_valid_fid,'%g\n',str2num(orbitnr_valid));
      fclose(orbitnr_valid_fid);

      btemp_valid_fid = fopen(['validation/btemp_valid.txt'],'w');
      fprintf(btemp_valid_fid,'%g\n',Btemp_valid);
      fclose(btemp_valid_fid);

      btemp_ifgs_valid_fid = fopen(['validation/btemp_ifgs_valid.txt'],'w');
      fprintf(btemp_ifgs_valid_fid,'%g\n',round(365*Btemp));
      fclose(btemp_ifgs_valid_fid);

      ifgs_valid_fid = fopen(['validation/ifgs_valid.txt'],'w');
      fprintf(ifgs_valid_fid,'%g\t%g\n',ifgs_valid');
      fclose(ifgs_valid_fid);

  end


  if max(abs(Btemp))>33 % to make sure Btemp is in years
```

```
    Btemp = Btemp/365;
  end

  if ischar(slc_selection_input)
    slc_selection = ps_determine_slc_selection(orbitnr,slc_selection_input);
  elseif isempty(slc_selection_input)
    slc_selection = 1:Nslc;
  else
    slc_selection = slc_selection_input;
  end

  if ~isempty(breakpoint)
    if breakpoint>1000
      breakpoint = find(str2num(orbitnr)==breakpoint);
    end
  end

  if ~isempty(breakpoint2)
    if breakpoint2>1000
      breakpoint2 = find(str2num(orbitnr)==breakpoint2);
    end
  end

  crop_final = [max(crop_in(:,1)) min(crop_in(:,2)) max(crop_in(:,3))
min(crop_in(:,4))];

  if ~isempty(crop)
    crop = [max(crop(1),crop_final(1)) min(crop(2),crop_final(2)) ...
            max(crop(3),crop_final(3)) min(crop(4),crop_final(4))];
  else
    crop = crop_final;
  end
  Nlines = crop(2)-crop(1)+1;
  Npixels = crop(4)-crop(3)+1;


  fid_res = fopen([project_id '_resfile.txt'],'w');
  fprintf(fid_res,'\n****************************************\n\n');
  fprintf(fid_res,['Persistent Scatterer analysis ' project_id ' \n']);
  %fprintf(fid_res,['Version: ' toolbox_version '\n']);
  fprintf(fid_res,'Result file\n');
  fprintf(fid_res,'%s\n\n',date);
  fprintf(fid_res,'****************************************\n\n');
  fprintf(fid_res,'Input file:               %s\n',input_file);
  fprintf(fid_res,'Number of interferograms: %3.0f\n',Nifgs);
  fclose(fid_res);

  save([project_id '_project.mat']);
  clear [^project_id ^processing_groups];
  pack

end


if find(processing_groups == 2)

  load([project_id '_project.mat']);
  ps_readinput_parameters; % script to read input parameters
  if isempty(processing_groups)
    processing_groups = 1:8;
  end
  if ~isempty(crop)
    crop = [max(crop(1),crop_final(1)) min(crop(2),crop_final(2)) ...
            max(crop(3),crop_final(3)) min(crop(4),crop_final(4))];
```

```
else
  crop = crop_final;
end

% ------------------------------------------------------------------------
% 2
% PSC selection
% ------------------------------------------------------------------------

fprintf(1,'\n');
fprintf(1,'SLC calibration....\n');

fid_res = fopen([project_id '_resfile.txt'],'a');
fprintf(fid_res,'\n');
fprintf(fid_res,[datestr(now) ', group2, start calibration ...\n']);
fclose(fid_res);

if strcmp(amplitude_calibration, 'yes')
  [calfactors] = ps_calibration(filenames_slc,...
                                filenames_ifgs,...
                                filename_water_mask,...
                                psc_selection_gridsize,...
                                slc_selection,...
                                crop,...
                                crop_in);
else
  calfactors = ones(1,length(slc_selection));
end

fid_res = fopen([project_id '_resfile.txt'],'a');
fprintf(fid_res,[datestr(now) ', group2, end calibration.\n']);

fprintf(1,'\n');
fprintf(1,'Initial PSC selection....\n');

fprintf(fid_res,[datestr(now) ', group2, start selection ...\n']);
fclose(fid_res);

[grid_array_az,...
 grid_array_r,...
 Npsc,...
 Npsp] = ps_selection(filenames_slc,...
                      filenames_ifgs,...
                      filenames_h2ph,...
                      psc_selection_method,...
                      psc_selection_gridsize,...
                      psc_threshold,...
                      Npsc_selections,...
                      psp_selection_method,...
                      psp_threshold1,...
                      psp_threshold2,...
                      slc_selection,...
                      Btemp,...
                      ps_area_of_interest,...
                      filename_water_mask,...
                      crop,...
                      crop_in,...
                do_apriori_sidelobe_mask,...
                      calfactors,...
                gamma_threshold,...
                psc_distribution,...
                      livetime_threshold,...
                      peak_tolerance,...
                      ref_cn);
```

```
  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,[datestr(now) ', group2, end selection.\n']);
  fclose(fid_res);

  save([project_id '_project.mat']);

  ps_selection_plots;

  clear [^project_id ^processing_groups];
  pack

end


if find(processing_groups == 3)

  load([project_id '_project.mat']);
  ps_readinput_parameters; % script to read input parameters
  if isempty(processing_groups)
    processing_groups = 1:8;
  end
  if ~isempty(crop)
    crop = [max(crop(1),crop_final(1)) min(crop(2),crop_final(2)) ...
            max(crop(3),crop_final(3)) min(crop(4),crop_final(4))];
  else
    crop = crop_final;
  end

  for z = 1:Npsc_selections
    copyfile([project_id '_psc_2orig_sel' num2str(z) '.raw'],[project_id
'_psc_sel' num2str(z) '.raw']);

    switch ps_eval_method
      case 'psp'
        copyfile([project_id '_psp_2orig_sel' num2str(z) '.raw'],[project_id
'_psp_sel' num2str(z) '.raw']);
    end
  end

  switch detrend_method
    case 'yes'

      fprintf(1,'\n');
      fprintf(1,'Trend removal....\n');

      % ------------------------------------------------------------------------
      % 3a
      % PS network
      % ------------------------------------------------------------------------

      fprintf(1,'\n');
      fprintf(1,'Network construction....\n');

      fid_res = fopen([project_id '_resfile.txt'],'a');
      fprintf(fid_res,'\n');
      fprintf(fid_res,[datestr(now) ', group3, start network construction
...\n']);
      fclose(fid_res);

      results_id = 'orig';

      [sig2_est,...
       Nref,...
```

```
       final_althyp_index] = ps_network(Nifgs,...
                                        Npsc,...
                                        Npsc_selections,...
                                        Nref,...
                                        max_arc_length,...
                                        psc_model,...
                                        final_model,...
                                        Btemp,...
                                        Bdop,...
                                        std_param,...
                                        breakpoint,...
                                        breakpoint2,...
                                        network_method,...
                                        Ncon,...
                                        Nparts,...
                                        ens_coh_threshold,...
                                        varfac_threshold,...
                                        ref_cn);

      fid_res = fopen([project_id '_resfile.txt'],'a');
      fprintf(fid_res,[datestr(now) ', group3, end network construction.\n']);
      fclose(fid_res);

      save([project_id '_project.mat']);

      ps_spatial_unwrapping_plots; % make plots of networks

      ps_visualize_psc(Nifgs,Npsc,Npsc_selections,final_model); % make plots of
psc


      % --------------------------------------------------------------------
      % 4
      % PS detrend (based on psc)
      % --------------------------------------------------------------------

      fprintf(1,'\n');
      fprintf(1,'Detrend (based on psc) ....\n');

      fid_res = fopen([project_id '_resfile.txt'],'a');
      fprintf(fid_res,[datestr(now) ', group3, start detrend ...\n']);
      fclose(fid_res);

      ps_detrend(Nifgs,...
                 Npsc,...
                 Npsp,...
                 Npsc_selections,...
                 filenames_output);

      fid_res = fopen([project_id '_resfile.txt'],'a');
      fprintf(fid_res,[datestr(now) ', group3, end detrend.\n']);
      fclose(fid_res);

      detrend_flag = 'yes';

    otherwise

      fprintf(1,'\n');
      fprintf(1,'No trend is removed....\n');

      detrend_flag = 'no';
      for z = 1:Npsc_selections
        copyfile([project_id '_psc_2orig_sel' num2str(z) '.raw'],[project_id
'_psc_3detr_sel' num2str(z) '.raw']);
```

```
        switch ps_eval_method
          case 'psp'
            copyfile([project_id '_psp_2orig_sel' num2str(z) '.raw'],[project_id
'_psp_3detr_sel' num2str(z) '.raw']);
        end
      end

  end

  save([project_id '_project.mat']);
  clear [^project_id ^processing_groups];
  pack

end


if find(processing_groups == 4)

  load([project_id '_project.mat']);
  ps_readinput_parameters; % script to read input parameters
  if isempty(processing_groups)
    processing_groups = 1:8;
  end
  if ~isempty(crop)
    crop = [max(crop(1),crop_final(1)) min(crop(2),crop_final(2)) ...
            max(crop(3),crop_final(3)) min(crop(4),crop_final(4))];
  else
    crop = crop_final;
  end

  for z = 1:Npsc_selections
    copyfile([project_id '_psc_3detr_sel' num2str(z) '.raw'],[project_id
'_psc_sel' num2str(z) '.raw']);

    switch ps_eval_method
      case 'psp'
        copyfile([project_id '_psp_3detr_sel' num2str(z) '.raw'],[project_id
'_psp_sel' num2str(z) '.raw']);
    end
  end

  % ---------------------------------------------------------------------
  % 3b
  % PS network
  % ---------------------------------------------------------------------

  fprintf(1,'\n');
  fprintf(1,'Network construction....\n');

  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,'\n');
  fprintf(fid_res,[datestr(now) ', group4, start network construction ...\n']);
  fclose(fid_res);

  if strcmp(detrend_flag,'yes')
    results_id = 'detr';
  elseif strcmp(detrend_flag,'no')
    results_id = 'orig';
  end

  [sig2_est,...
   Nref,...
   final_althyp_index] = ps_network(Nifgs,...
```

```
                            Npsc,...
                            Npsc_selections,...
                            Nref,...
                            max_arc_length,...
                            psc_model,...
                            final_model,...
                            Btemp,...
                            Bdop,...
                            std_param,...
                            breakpoint,...
                            breakpoint2,...
                            network_method,...
                            Ncon,...
                            Nparts,...
                            ens_coh_threshold,...
                                    varfac_threshold,...
                            ref_cn);

fid_res = fopen([project_id '_resfile.txt'],'a');
fprintf(fid_res,[datestr(now) ', group4, end network construction.\n']);
fclose(fid_res);

save([project_id '_project.mat']);

ps_spatial_unwrapping_plots; % make plots of networks
ps_visualize_psc(Nifgs,Npsc,Npsc_selections,final_model); % make plots of psc


% --------------------------------------------------------------------
% 5a
% PS filtering
% --------------------------------------------------------------------

fprintf(1,'\n');
fprintf(1,'Separation atmosphere and unmodeled deformation ....\n');

fid_res = fopen([project_id '_resfile.txt'],'a');
fprintf(fid_res,[datestr(now) ', group4, start filtering ...\n']);
fclose(fid_res);

ps_filtering(Btemp,...
           ts_atmo_filter,...
           ts_atmo_filter_length,...
           Nifgs,...
           Npsc,...
           Npsc_selections);

fid_res = fopen([project_id '_resfile.txt'],'a');
fprintf(fid_res,[datestr(now) ', group4, end filtering.\n']);
fclose(fid_res);



% --------------------------------------------------------------------
% 5b
% Atmosphere estimation by Kriging
% --------------------------------------------------------------------

fprintf(1,'\n');
fprintf(1,'Atmosphere estimation by Kriging....\n');

fid_res = fopen([project_id '_resfile.txt'],'a');
fprintf(fid_res,[datestr(now) ', group4, start atmosphere estimation ...\n']);
fclose(fid_res);
```

```
  ps_atmo_kriging(Nifgs,...
              Npsc,...
              Npsc_selections,...
              Npsp,...
              filenames_output);

  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,[datestr(now) ', group4, end atmosphere estimation.\n']);
  fclose(fid_res);

  ps_visualize_atmo(Nifgs,Npsp,Npsc,Npsc_selections,dates);

  save([project_id '_project.mat']);
  clear [^project_id ^processing_groups];
  pack

end


if find(processing_groups == 5)

  load([project_id '_project.mat']);
  ps_readinput_parameters; % script to read input parameters
  if isempty(processing_groups)
    processing_groups = 1:8;
  end
  if ~isempty(crop)
    crop = [max(crop(1),crop_final(1)) min(crop(2),crop_final(2)) ...
            max(crop(3),crop_final(3)) min(crop(4),crop_final(4))];
  else
    crop = crop_final;
  end

  for z = 1:Npsc_selections
    copyfile([project_id '_psc_4atmo_sel' num2str(z) '.raw'],[project_id
'_psc_sel' num2str(z) '.raw']);

    switch ps_eval_method
      case 'psp'
        copyfile([project_id '_psp_4atmo_sel' num2str(z) '.raw'],[project_id
'_psp_sel' num2str(z) '.raw']);
    end
  end

  % ----------------------------------------------------------------------
  % 6
  % Make subset of available ifgs (optional)
  % ----------------------------------------------------------------------

  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,'\n');
  fclose(fid_res);

  if isempty(ifg_selection_input)
    % do nothing
  else

    fid_res = fopen([project_id '_resfile.txt'],'a');
    fprintf(fid_res,[datestr(now) ', group5, start interferogram selection
...\n']);
    fclose(fid_res);

    if exist('Nifgs_orig')
```

```
      Nifgs = Nifgs_orig;
      Btemp = Btemp_orig;
      Bdop = Bdop_orig;
      Bperp = Bperp_orig;
    else
      Nifgs_orig = Nifgs;
      Btemp_orig = Btemp;
      Bdop_orig = Bdop;
      Bperp_orig = Bperp;
    end

    if ischar(ifg_selection_input) %
      ifg_selection =
ps_determine_ifg_selection(orbitnr(1:Nifgs,:),ifg_selection_input);
    else
      ifg_selection = ifg_selection_input;
    end
    Nifgs = ps_select_ifgs(ifg_selection,Npsc,Npsc_selections,Npsp,Nifgs);
    Btemp = Btemp(ifg_selection);
    Bdop = Bdop(ifg_selection);
    Bperp = Bperp(ifg_selection);

    fid_res = fopen([project_id '_resfile.txt'],'a');
    fprintf(fid_res,[datestr(now) ', group5, end interferogram selection.\n']);
    fclose(fid_res);

  end



  % ------------------------------------------------------------------------
  % 3c
  % PS network
  % ------------------------------------------------------------------------

  fprintf(1,'\n');
  fprintf(1,'Network construction....\n');

  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,[datestr(now) ', group5, start network construction ...\n']);
  fclose(fid_res);

  results_id = 'atmo';

  [sig2_est,...
   Nref,...
   final_althyp_index] = ps_network(Nifgs,...
                                    Npsc,...
                                    Npsc_selections,...
                                    Nref,...
                                    max_arc_length,...
                                    psc_model,...
                                    final_model,...
                                    Btemp,...
                                    Bdop,...
                                    std_param,...
                                    breakpoint,...
                                    breakpoint2,...
                                    network_method,...
                                    Ncon,...
                                    Nparts,...
                                    ens_coh_threshold,...
                                    varfac_threshold,...
                                    ref_cn);
```

```
fid_res = fopen([project_id '_resfile.txt'],'a');
fprintf(fid_res,[datestr(now) ', group5, end network construction.\n']);
fclose(fid_res);

save([project_id '_project.mat']);

ps_spatial_unwrapping_plots; % make plots of networks
ps_visualize_psc(Nifgs,Npsc,Npsc_selections,final_model); % make plots of psc


% ------------------------------------------------------------------------
% 7a
% PS densification after atmosphere correction
% ------------------------------------------------------------------------

fprintf(1,'\n');
fprintf(1,['PS densification by ' ps_method ' ....\n']);

fid_res = fopen([project_id '_resfile.txt'],'a');
fprintf(fid_res,[datestr(now) ', group5, start densification ...\n']);
fclose(fid_res);

Nps_atmo = ps_densification(Btemp,...
                            Bdop,...
                            grid_array_az,...
                            grid_array_r,...
                            Nifgs,...
                            Npsc,...
                            Npsc_selections,...
                            Npsp,...
                            Nref,...
                            ps_model,...
                            final_model,...
                            final_althyp_index,...
                            std_param,...
                            sig2_est,...
                            breakpoint,...
                            breakpoint2,...
                            Namp_disp_bins,...
                            Ndens_iterations,...
                            densification_flag,...
                            filenames_output,...
                            filenames_ifgs,...
                            filenames_h2ph,...
                            detrend_flag,...
                            defo_model_flag,...
                            ps_area_of_interest,...
                            dens_method,...
                            dens_check,...
                            filename_water_mask,...
                            crop,...
                            crop_in,...
                            Nest,...
                            master_index,...
                            Nslc);

fid_res = fopen([project_id '_resfile.txt'],'a');
fprintf(fid_res,[datestr(now) ', group5, end densification.\n']);
fclose(fid_res);

save([project_id '_project.mat']);

ps_visualize_ps(Nifgs,Nps_atmo,Npsc,Npsc_selections,final_model); % make plots
```

```
of ps

  save([project_id '_project.mat']);
  clear [^project_id ^processing_groups];
  pack

end


if find(processing_groups == 6)

  load([project_id '_project.mat']);
  ps_readinput_parameters; % script to read input parameters
  if isempty(processing_groups)
    processing_groups = 1:8;
  end
  if ~isempty(crop)
    crop = [max(crop(1),crop_final(1)) min(crop(2),crop_final(2)) ...
            max(crop(3),crop_final(3)) min(crop(4),crop_final(4))];
  else
    crop = crop_final;
  end

  if ~isempty(defo_method)

    % ----------------------------------------------------------------------
    % 8
    % Estimation of the deformation model
    % ----------------------------------------------------------------------

    fprintf(1,'\n');
    fprintf(1,['Estimation of deformation model....\n']);

    fid_res = fopen([project_id '_resfile.txt'],'a');
    fprintf(fid_res,'\n');
    fprintf(fid_res,[datestr(now) ', group6, start deformation modeling
...\n']);
    fclose(fid_res);

    ps_defo_model(Btemp,...
                  Nifgs,...
                  Npsc,...
                  Npsp,...
                  Nps_atmo,...
                  Npsc_selections,...
                  std_param,...
                  xc0,...
                  yc0,...
                  zc0,...
                  r0,...
                  r10,...
                  epoch,...
                  defo_method,...
                  filenames_output);

    fid_res = fopen([project_id '_resfile.txt'],'a');
    fprintf(fid_res,[datestr(now) ', group6, end deformation modeling.\n']);
    fclose(fid_res);

    defo_model_flag = 'yes';

    for z = 1:Npsc_selections
      copyfile([project_id '_psc_5defo_sel' num2str(z) '.raw'],[project_id
'_psc_sel' num2str(z) '.raw']);
```

```
      switch ps_eval_method
        case 'psp'
          copyfile([project_id '_psp_5defo_sel' num2str(z) '.raw'],[project_id
'_psp_sel' num2str(z) '.raw']);
      end
    end

    % ----------------------------------------------------------------------
    % 3d
    % PS network
    % ----------------------------------------------------------------------

    fprintf(1,'\n');
    fprintf(1,'Network construction....\n');

    fid_res = fopen([project_id '_resfile.txt'],'a');
    fprintf(fid_res,[datestr(now) ', group6, start network construction
...\n']);
    fclose(fid_res);

    results_id = 'defo';

    [sig2_est,...
     Nref,...
     final_althyp_index] = ps_network(Nifgs,...
                                      Npsc,...
                                      Npsc_selections,...
                                      Nref,...
                                      max_arc_length,...
                                      psc_model,...
                                      final_model,...
                                      Btemp,...
                                      Bdop,...
                                      std_param,...
                                      breakpoint,...
                                      breakpoint2,...
                                      network_method,...
                                      Ncon,...
                                      Nparts,...
                                      ens_coh_threshold,...
                                      varfac_threshold,...
                                      ref_cn);

    fid_res = fopen([project_id '_resfile.txt'],'a');
    fprintf(fid_res,[datestr(now) ', group6, end network construction.\n']);
    fclose(fid_res);

    save([project_id '_project.mat']);

    ps_spatial_unwrapping_plots; % make plots of networks
    ps_visualize_psc(Nifgs,Npsc,Npsc_selections,final_model); % make plots of
psc


    % ----------------------------------------------------------------------
    % 7b
    % PS densification after defo correction
    % ----------------------------------------------------------------------

    fprintf(1,'\n');
    fprintf(1,['PS densification by ' ps_method ' ....\n']);

    fid_res = fopen([project_id '_resfile.txt'],'a');
```

```
    fprintf(fid_res,[datestr(now) ', group6, start densification ...\n']);
    fclose(fid_res);

    Nps_defo = ps_densification(Btemp,...
                                Bdop,...
                                grid_array_az,...
                                grid_array_r,...
                                Nifgs,...
                                Npsc,...
                                Npsc_selections,...
                                Npsp,...
                                Nref,...
                                ps_model,...
                                final_model,...
                                final_althyp_index,...
                                std_param,...
                                sig2_est,...
                                breakpoint,...
                                breakpoint2,...
                                Namp_disp_bins,...
                                Ndens_iterations,...
                                densification_flag,...
                                filenames_output,...
                                filenames_ifgs,...
                                filenames_h2ph,...
                                detrend_flag,...
                                defo_model_flag,...
                                ps_area_of_interest,...
                                dens_method,...
                                dens_check,...
                                filename_water_mask,...
                                crop,...
                                crop_in,...
                                Nest,...
                                master_index,...
                                Nslc);

    fid_res = fopen([project_id '_resfile.txt'],'a');
    fprintf(fid_res,[datestr(now) ', group6, end densification.\n']);
    fclose(fid_res);

    save([project_id '_project.mat'])

    ps_visualize_ps(Nifgs,Nps_defo,Npsc,Npsc_selections,final_model);

  else

    defo_model_flag = 'no';
    Nps_defo = [];

  end

  save([project_id '_project.mat']);
  clear [^project_id ^processing_groups];
  pack

end


if find(processing_groups == 7)

  load([project_id '_project.mat']);
  ps_readinput_parameters; % script to read input parameters
  if isempty(processing_groups)
```

```
    processing_groups = 1:8;
  end
  if ~isempty(crop)
    crop = [max(crop(1),crop_final(1)) min(crop(2),crop_final(2)) ...
            max(crop(3),crop_final(3)) min(crop(4),crop_final(4))];
  else
    crop = crop_final;
  end

  % ------------------------------------------------------------------------
  % 9
  % Output generation
  % ------------------------------------------------------------------------

  fprintf(1,'\n');
  fprintf(1,['Calculate spatio-temporal consistency ....\n']);

  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,'\n');
  fprintf(fid_res,[datestr(now) ', group7, start spatio-temporal
consistency ...\n']);
  fclose(fid_res);

  ps_spatio_temporal_consistency(Nifgs,...
                        Nps_atmo,...
                        Nps_defo,...
                        Npsc_selections,...
                        defo_model_flag,...
                        stc_min_max,...
                        'orig',...
                        ts_noise_filter,...
                        ts_noise_filter_length);

  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,[datestr(now) ', group7, end spatio-temporal
consistency.\n']);
  fclose(fid_res);

  fprintf(1,'\n');
  fprintf(1,['Create output ....\n']);

  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,[datestr(now) ', group7, start output ...\n']);
  fclose(fid_res);


ps_output(Nifgs,Nps_atmo,Nps_defo,Npsc_selections,final_model,defo_model_flag,ma
ster_res,ref_height,output_format,dates,Btemp,do_aposteriori_sidelobe_mask,crop)
;

  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,[datestr(now) ', group7, end output.\n']);
  fclose(fid_res);

  switch ps_eval_method
    case 'whole'
     switch densification_flag
      case 'yes'

       fid_res = fopen([project_id '_resfile.txt'],'a');
       fprintf(fid_res,[datestr(now) ', group7, start output no
densification ...\n']);
       fclose(fid_res);
```

```
       [Nps_atmo_nodens,Nps_defo_nodens] =
ps_output_nodens(Nifgs,Nps_atmo,Nps_defo,Npsc_selections,final_model,defo_model_
flag,master_res,ref_height,output_format,dates,Btemp,do_aposteriori_sidelobe_mas
k,crop);

       fid_res = fopen([project_id '_resfile.txt'],'a');
       fprintf(fid_res,[datestr(now) ', group7, end output no
densification.\n']);
       fclose(fid_res);


ps_visualize_ps_nodens(Nifgs,Nps_atmo_nodens,Npsc,Npsc_selections,final_model);
       if ~isempty(defo_method)

ps_visualize_ps_nodens(Nifgs,Nps_defo_nodens,Npsc,Npsc_selections,final_model);
       end
     end
  end

  fclose('all');

  save([project_id '_project.mat']);
  clear [^project_id ^processing_groups];
  pack

end

if find(processing_groups == 8)

  load([project_id '_project.mat']);
  ps_readinput_parameters; % script to read input parameters
  if isempty(processing_groups)
    processing_groups = 1:8;
  end
  if ~isempty(crop)
    crop = [max(crop(1),crop_final(1)) min(crop(2),crop_final(2)) ...
            max(crop(3),crop_final(3)) min(crop(4),crop_final(4))];
  else
    crop = crop_final;
  end

  % ------------------------------------------------------------------------
  % 10
  % Time series filtering
  % ------------------------------------------------------------------------

  fprintf(1,'\n');
  fprintf(1,['Filtering time series ....\n']);

  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,'\n');
  fprintf(fid_res,[datestr(now) ', group8, start noise filtering ...\n']);
  fclose(fid_res);

  ps_noise_filter(Nifgs,...
              Nps_atmo,...
              Nps_defo,...
              Npsc_selections,...
              Btemp,...
              Bdop,...
              defo_model_flag,...
              final_model,...
              final_althyp_index,...
              std_param,...
```

```
              sig2_est,...
              ts_noise_filter,...
              ts_noise_filter_length);

  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,[datestr(now) ', group8, end noise filtering.\n']);
  fclose(fid_res);

  fprintf(1,'\n');
  fprintf(1,['Calculate spatio-temporal consistency filtered....\n']);

  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,[datestr(now) ', group8, start spatio-temporal
consistency ...\n']);
  fclose(fid_res);

  ps_spatio_temporal_consistency(Nifgs,...
                         Nps_atmo,...
                         Nps_defo,...
                         Npsc_selections,...
                         defo_model_flag,...
                         stc_min_max,...
                         'filt',...
                         ts_noise_filter,...
                         ts_noise_filter_length);

  fid_res = fopen([project_id '_resfile.txt'],'a');
  fprintf(fid_res,[datestr(now) ', group8, end spatio-temporal
consistency.\n']);
  fclose(fid_res);

  fclose('all');

  save([project_id '_project.mat']);
  clear all

end

fprintf(1,'\n');
fprintf(1,'Done!\n');


% ------------------------------------------------------------------------
% The end
% ------------------------------------------------------------------------
```